

Document number: P0534R2
Date: 2017-07-31
Reply-to: Oliver Kowalke (oliver.kowalke@gmail.com)
Authors: Oliver Kowalke (oliver.kowalke@gmail.com), Nat Goodspeed (nat@lindenlab.com)
Audience: LEWG

call/cc (call-with-current-continuation): A low-level API for stackful context switching

Abstract	1
Why should <i>call/cc</i> be standardized?	2
Revision History	2
Continuations	3
Call with current continuation	4
Design	6
Performance of <i>call/cc</i>	12
API	13
Additional notes	19
References	20

Abstract

This document proposes a C++ equivalent to the well-known concept **call-with-current-continuation** (abbreviated **call/cc**). This facility permits a program written in portable C++ to subdivide processing into distinct **contexts**: units smaller than a thread.

Within this proposal, the unadorned term “thread” means a `std::thread` (or **kernel thread**). When the Standard’s more general term “thread of execution” is intended, it is spelled out in full.

With *call/cc*, processing in a given thread may be further subdivided into multiple contexts. Each such context qualifies as a “thread of execution” according to the definition in the Standard. However, within a given thread, control is cooperatively passed from one context to another.

This has a couple of important implications:

- In each thread in a process, exactly one context is running at any given time. All others are **suspended**.
- The running context on a thread continues running until it explicitly **resumes** some other context. The act of resuming another context suspends the previously-running context. This transfer of control, in which one context suspends and another resumes, is **context-switching**.
- There are no data races between contexts running on the same thread.

The kind of context-switching presented in this proposal is called **stackful** because each context requires some implementation of the C++ stack. C++ code running on a particular context may transparently call ordinary C++ functions. In contrast to the `co_await` facility (proposed separately⁴), this permits encapsulation. A function that suspends (by resuming some other context) needs no special signature. Its caller need not be aware that it might suspend. It need not call that function in any special way.

This supports use cases that cannot be addressed with `co_await` alone.

Also in contrast to the `co_await` facility, this proposal requires no changes to the core C++ language. *call/cc* is presented as a library facility, albeit a library that cannot be implemented in portable C++. This is why it is desirable to incorporate it into the International Standard.

Consider the following bullets from P0559R0:⁶

- Avoid ‘compiler magic’ when possible
- Prefer library solutions over language changes if feasible

The proposed *call/cc* facility is intended to be foundational. While of course application coders are free to use the *call/cc* API, its real promise is in supporting higher-level abstractions.

This proposal describes the basic *call/cc* facility, presents some illustrative use cases and explains why the API is set at its present level.

Why should *call/cc* be standardized?

The *call/cc* facility cannot itself be implemented in portable C++. The present implementation,¹² maintained by a single author, supports a small set of current platforms available to that author. Should *call/cc* be integrated into the Standard, it will become universally available.

Moreover, correct support for certain platforms might involve undocumented complexity. The runtime vendor is best positioned to implement the specified functionality.

Compiler awareness of this facility could enable certain optimizations as well:

- The compiler might be able to analyze the code to be launched on a new *call/cc* context and determine an optimal stack size for that context.
- The compiler might be able to determine that not all registers need be preserved across a context switch.
- For certain use cases, the compiler might be able to optimize away context-switching altogether. Promising work has been done in this area for the *co_await* facility.⁴

Revision History

This document supersedes P0534R1.⁵

Changes since P0534R1:

- API modified
 - `any_thread()` removed
- `std::continuation` may only be resumed on the same `std::thread` on which it was created
- only code running on a `std::thread` (e.g. the thread implicitly created for `main()`) or an execution agent created by `std::callcc()` may call `std::callcc()`
- illustrative use cases removed

Continuations

A continuation is an abstract concept that represents the context state at a given point during the execution of a program. That implies that a continuation represents the remaining steps of a computation.

As a **basic, low-level primitive** it can be used to implement control structures like coroutines, generators, lightweight threads, cooperative multitasking (fibers), backtracking, non-deterministic choice. In classic event-driven programs, organized around a main loop that fetches and dispatches incoming I/O events, certain asynchronous I/O sequences are logically sequential. Use of continuations permits writing and maintaining code that looks and acts sequential, even though from time to time it may suspend while asynchronous I/O is pending.

C and C++ already use implicit continuations: when running code calls a function, then a (hidden) continuation (the remaining steps after the function call) is created. This continuation is resumed when the function returns. For instance the x86 architecture stores the (hidden) continuation as a return address on the stack.*

Continuations exposed as **first-class continuations** can be passed to and returned from functions, assigned to variables or stored into containers. With first-class continuations, a language can explicitly **control the flow of execution** by suspending and resuming continuations, enabling control to pass into a function at exactly the point where it previously suspended. Making the program state visible via first-class continuations is known as **reification**.

The remainder of the computation derived from the current point in a program's execution is called the **current continuation**. *call/cc* captures the **current continuation** and passes it to the function invoked by *call/cc*.

Continuations that can be called multiple times are named **full continuations**.

One-shot continuations can only be resumed once: once resumed, a **one-shot continuation** is invalidated.

Full continuations are **not** considered in this proposal because of their nature, which is problematic in C++. Full continuations would require copies of the stack (including all stack variables), which would violate C++'s RAII pattern.

In contrast to *call/cc* that captures the **entire remaining** continuation, the operators *shift* and *reset* create a so-called **delimited continuation**. A delimited continuation represents a slice of the program context. Operator *reset* delimits the continuation, i.e. it determines where the continuation starts and ends, while *shift* **reifies** the continuation.

Delimited continuations are **not** part of this proposal. However, delimited continuation functionality can be built on *call/cc*.

*Other (RISC) architectures use a special micro-processor register for this purpose.

Call with current continuation

call/cc (abbreviation of 'call with current continuation') is a universal control operator (well-known from languages like Scheme, Ruby, Lisp ...) that captures the **current continuation** (the sequence of instructions after *call/cc* returns) as a **first-class object** and passes it to a function that is executed in a newly-created execution context.

`std::callcc()` is the C++ equivalent to *call/cc*, preserving the **call state** and the **program state** (variables).

When code running in some *original* context calls `resume()` on some `std::continuation` instance `target`, the *original* context is saved and the `target` continuation is restored in its place, so that program flow will continue at the point at which the `target` continuation was originally captured. The captured *original* continuation then becomes the *return value* of the `std::callcc()` invocation in `target`.

`std::continuation` is a **one-shot continuation**: it can be resumed at most once, is only move-constructible and move-assignable.

```
std::continuation foo(std::continuation && caller) {
    while (caller) {
        std::cout << "foo\n";
        caller= // (4)
            caller.resume(); // (1)
    }
    return std::move(caller);
}
```

```
std::continuation foo_ct= // (2)
    std::callcc(foo); // (0)
while (foo_ct) {
    std::cout << "bar\n";
    foo_ct= // (5)
        foo_ct.resume(); // (3)
}
```

```
output:
foo
bar
...
```

The `std::callcc(foo)` call at (0) captures the **current continuation**, entering function `foo()` while passing the captured continuation as argument `caller`.

As long as continuation `caller` is valid, "foo" is passed to standard output.

The expression `caller.resume()` at (1) resumes the original continuation represented within `foo()` by `caller` and transfers back the control of execution to `main()`. On return from `std::callcc(foo)`, the assignment at (2) sets `foo_ct` to the **current continuation** as of (1).

The call to `foo_ct.resume()` at (3) resumes function `foo`, returning from the `resume()` call at (1) and executing the assignment at (4). Here we replace the `std::continuation` instance `caller` invalidated by the `resume()` call at (1) with the new instance returned by that same `resume()` call.

Function `std::callcc()` captures the **current continuation** and enters the given function immediately, while `resume()` returns control back to the continuation saved in **this*.

The presented code prints out "foo" and "bar" in a endless loop.

In order to transfer data, `std::callcc()` and `resume()` accept arguments. These are stored on the stack of the captured **current continuation**. Function `data_available()` tests whether data have been passed, and with `get_data()` the data can be retrieved.

```
std::continuation lambda=
    std::callcc( // (0)
        [](std::continuation && caller){
            int a=0;
```

```

        int b=1;
        for(;;){
            caller=caller.resume(a); // (1)
            int next=a+b;
            a=b;
            b=next;
        }
        return std::move(caller);
    });
for (int j=0;j<10;++j) {
    int i=lambda.get_data<int>(); // (2)
    std::cout << i << " ";
    lambda=lambda.resume(); // (3)
}

```

output:

```
0 1 1 2 3 5 8 13 21 34 55
```

The invocation of `std::callcc()` at (0) immediately enters the lambda, passing no data but the **current continuation**. The lambda calculates the fibonacci number using local variables `a`, `b` and `next`. The calculated fibonacci number is transferred via `resume()` at (1). The execution control returns; `lambda` now represents the continuation of the lambda. With `get_data()` at (2) the fibonacci number is transferred to the current context while at (3) the lambda is entered again in order to compute the next fibonacci number – without passing any parameter to the lambda.

Design

std::callcc() as a factory function Every valid `std::continuation` instance is synthesized by the `std::callcc()` facility:

- as a parameter passed into the function called by `std::callcc()` or `resume_with()`
- as the value returned by `std::callcc()`, `resume()` or `resume_with()`.

This is intentional for consistency with the *call/cc* facility in other languages.^{7,8}

Footprint `std::continuation` contains only its **stack pointer** and a data pointer as member variables. It should typically be no larger than two pointers.

Passing data Data may be passed to another context as additional arguments of `std::callcc()` and `resume()`.*

With functions `data_available()` and `get_data()` the code can test for data and, if desired, retrieve the data.

Any additional data arguments passed to `std::callcc()` or `resume()` can be retrieved by the created/resumed context using `get_data()`. The template arguments for `get_data()` must agree with the types passed to `std::callcc()` or `resume()`:

- If you call `std::callcc(fn)` or `resume()` with no additional arguments, then `data_available()` will return **false**; `fn()` may not call `get_data()` at all.
- If you call `std::callcc(fn, single_arg)` or `resume(single_arg)`, then `data_available()` will return **true**, and if `fn()` calls `get_data()`, its template argument must match `get_data<decltype(single_arg)>()`.
- If you call `std::callcc(fn, multiple_args...)` or `resume(multiple_args...)`, then `data_available()` will return **true**, and if `fn()` calls `get_data()`, its template arguments must match the types of `multiple_args...`. Specifically, `get_data<Args...>()` returns `std::tuple<Args...>`; that `std::tuple` must be compatible with `std::make_tuple(multiple_args...)`.

```
int i=1;
std::continuation lambda=
    std::callcc( // (0)
        [](std::continuation && caller){
            int j=caller.get_data<int>(); // (1)
            std::cout << "inside lambda, j==" << j << std::endl;
            caller=caller.resume(j+1); // (2)
            return std::move(caller); // (5)
        },
        i);
i=lambda.get_data<int>(); // (3)
std::cout << "i==" << i << std::endl;
lambda=lambda.resume(); // (4)
```

output:

```
inside lambda, j==1
i==2
```

The `callcc()` call at (0) enters the lambda and passes 1 into the new context. The value is retrieved as `j`, as shown by (1). The expression `caller.resume(j+1)` at (2) resumes the original context (represented within the lambda by `caller`) and transfers back an integer of `j+1`. The assignment at (3) sets `i` to `j+1`.

The call to `lambda.resume()` at (4) (note that no data is passed) resumes the lambda, returning from the `caller.resume(j+1)` call at (2). Here, too, we replace the `std::continuation` instance `caller` invalidated by the `resume()` call at (2) with the new instance returned by that same `resume()` call.

Finally the lambda returns (the updated) `caller` at (5), terminating its context.

Since the updated `caller` represents the continuation suspended by the call at (4), control returns to `main()`.

*or returned from the function invoked by `resume_with()`; see section [Inject function into suspended context](#)

However, since context `lambda` has now terminated, the updated `lambda` is invalid. Its `operator bool()` returns `false`; its `operator!()` returns `true`.

Multiple arguments can be transferred into another continuation too.

```
int i=1, j=2;
std::continuation lambda=
    std::callcc( // (0)
        [](std::continuation && caller){
            auto [i, j]=caller.get_data<int, int>(); // (1)
            std::cout << "inside lambda, i==" << i << ", j==" << j << std::endl;
            caller=caller.resume(i+j); // (2)
            return std::move(caller); // (5)
        },
        i,
        j);
int k=lambda.get_data<int>(); // (3)
std::cout << "k==" << k << std::endl;
lambda=lambda.resume(); // (4)
```

output:

```
inside lambda, i==1, j==2
k==3
```

`caller.get_data<int, int>()` returns a `std::tuple<int, int>` containing the values passed by the `std::callcc()` call at (0).

main() and thread functions `main()` as well as the *entry-function* of a thread can be represented by a continuation. That `std::continuation` instance is synthesized when the running context suspends, and is passed into the newly-resumed context.

```
int main() {
    std::continuation lambda=
        std::callcc( // (0)
            [](std::continuation && caller){ // (1)
                return std::move(caller); // (2)
            });
    return 0;
}
```

The `callcc()` call at (0) enters the lambda. The `std::continuation` caller at (1) represents the execution context of `main()`. Returning caller at (2) resumes the original context, switching back to `main()`.

call/cc and std::thread Any context represented by a valid `std::continuation` instance is necessarily suspended.

It is only valid to resume a `std::continuation` instance on the thread on which it was initially launched.

Termination There are a few different ways to terminate a given context without terminating the whole process, or engaging undefined behavior.

- Return a valid continuation from the *entry-function*.
- Call `std::unwind_context()` with a valid continuation. This throws a `std::unwind_exception` instance that binds that continuation.
- [LEWG: Should we publish the `std::unwind_exception` constructor that accepts `std::continuation`? Then another supported way would be to construct and throw `std::unwind_exception` "by hand," which is what `std::unwind_context()` does internally.]
- Call `std::continuation::resume_with(std::unwind_context)`. This is what `~continuation()` does. Since `std::unwind_context()` accepts a `std::continuation`, and since `resume_with()` synthesizes a `std::continuation` and passes it to the subject function, this terminates the context referenced by the original `std::continuation` instance and switches back to the caller.
- Engage `~continuation()`: switch to some other context, which will receive a `std::continuation` instance representing the current context. Make that other context destroy the received `std::continuation` instance.

When the *entry-function* invoked by `std::callcc()` returns a valid `std::continuation` instance, the running context is terminated. Control switches to the context indicated by the returned `std::continuation` instance.

Returning an invalid `std::continuation` instance (`operator bool()` returns `false`) invokes undefined behavior.

If the *entry-function* returns the same `std::continuation` instance it was originally passed (or rather, the most recently updated `std::continuation` returned from `std::callcc()` or the previous instance's `resume()`), control returns to the context that most recently resumed the running context. However, the *entry-function* may return (switch to) any reachable valid `std::continuation` instance.

Calling `std::continuation::resume()` means: “Please switch to the indicated context; I am suspending; please resume me later.”

Returning a particular `std::continuation` means: “Please switch to the indicated context; and by the way, I am done.”

Exceptions In general, if an uncaught exception escapes from the *entry-function*, `std::terminate` is called. There is one exception: `std::unwind_exception`. The `std::callcc()` facility internally uses `std::unwind_exception` to clean up the stack of a suspended context being destroyed. This exception must be allowed to propagate out of an *entry-function*.

A correct *entry-function* `try / catch` block looks like this:

```
try {
    // ... body of context logic ...
} catch (std::unwind_exception const&) {
    // do not swallow unwind_exception
    throw;
} catch (...) {
    // ... log, or whatever ...
}
```

Of course, if you do not expect the *entry-function* or anything it calls to throw exceptions, you need no `try / catch` block.

If a `resume_with()` function throws an exception that you expect to catch in the context's *entry-function*, it is good practice to bind into the exception object the continuation passed to the `resume_with()` function so that the *entry-function*'s `catch` clause can return that continuation.

Inject function into suspended context Sometimes it is useful to inject a new function (for instance, to throw an exception) into a suspended context. For this purpose you may call `resume_with(Fn && fn, Args ... args)`, passing:

- the function `fn()` to execute
- additional data `args` to be retrieved by `fn()`.

Let's say that the context represented by the `std::continuation` instance `ctx` has suspended in a function `suspender()`. You intend to inject function `fn()` into context `ctx` as if `suspender()` had directly called `fn()` at the point where it suspended.

Like an *entry-function* passed to `std::callcc()`, `fn()` must accept `std::continuation&&`. However, instead of returning `std::continuation`, `fn()` must return a type corresponding to the `args` passed to the `resume_with()` call.

- If you call `ctx.resume_with(fn)` with no additional `args`, the return type of `fn()` is irrelevant: its return value is discarded.
- If you call `ctx.resume_with(fn, single_arg)`, then `fn()` must return `decltype(single_arg)`.
- If you call `ctx.resume_with(fn, multiple_args...)`, then `fn()` must return a `std::tuple` with corresponding types: specifically, `decltype(std::make_tuple(multiple_args...))`.

Any `args...` passed to `resume_with()` can be retrieved within `fn()` using `get_data()`. As with `std::callcc()` and `resume()`, the template arguments for `get_data()` must agree with the types passed to `resume_with()`:

- If you call `ctx.resume_with(fn)` with no additional `args`, then `data_available()` will return `false`: `fn()` may not call `get_data()` at all.

- If you call `ctx.resume_with(fn, single_arg)`, then `data_available()` will return `true`, and if `fn()` calls `get_data()`, its template argument must match `get_data<decltype(single_arg)>()`.
- If you call `ctx.resume_with(fn, multiple_args...)`, then `data_available()` will return `true`, and if `fn()` calls `get_data()`, its template arguments must match the types of `multiple_args...`. Specifically, `get_data<Args...>()` returns `std::tuple<Args...>`; that `std::tuple` must be compatible with `std::make_tuple(multiple_args...)`.

The above describes retrieving passed values *within* `fn()`. The value returned by `fn()` becomes available to `suspender()`. It is up to `fn()` whether to return the same value(s) it retrieved from the `resume_with()` call.

- If you call `ctx.resume_with(fn)` with no additional args, then `data_available()` will return `false` even within `suspender():suspender()` may not call `get_data()` at all.
- If you call `ctx.resume_with(fn, args...)` – with one or more args... – then `data_available()` will return `true` within `suspender().get_data()` within `suspender()` will retrieve the value returned by `fn()`.

Note that if you pass one or more `args...` to `resume_with()`, those `args...` *must* be compatible with the return type of `fn()`. However, it is permissible to call `resume_with(fn)` sometimes (`data_available()` returns `false`) but pass compatible data in other `resume_with()` calls.

Suppose that code running on the program's main context calls `std::callcc(f)`, thereby entering `f()`. This is the point at which `mc` is synthesized and passed into `f()`, as illustrated below.

Suppose further that after doing some work, `f()` calls `mc.resume()`, thereby switching back to the main context. `f()` remains suspended in the call to `mc.resume()`.

At this point the main context calls `f_ct.resume_with(g)` where `g()` is declared as `int g(continuation &&); g()` is entered in the context of `f()`. It is as if `f()`'s call to `mc.resume()` directly called `g()`.

Function `g()` has almost the same range of possibilities as any function called on `f()`'s context. Its special invocation matters when control leaves it in either of two ways:

1. If `g()` throws an exception, that exception unwinds all previous stack entries in that context (such as `f()`'s) as well, back to a matching `catch` clause.*
2. If `g()` returns, its return value provides data for `f()`'s suspended `mc.resume()` call.

```
std::continuation f(std::continuation && mc) {
    int data=mc.get_data<int>(); // (1)
    std::cout << "f: entered first time: " << data << std::endl;
    mc = // (5)
        mc.resume(data+1); // (2)
    data=mc.get_data<int>();
    std::cout << "f: entered second time: " << data << std::endl;
    mc = // (10)
        mc.resume(data+1); // (6)
    data=mc.get_data<int>(); // (11)
    std::cout << "f: entered third time: " << data << std::endl;
    return std::move(mc); // (12)
}

int g(std::continuation && mc) {
    int data=mc.get_data<int>();
    std::cout << "g: entered: " << data << std::endl;
    return -1; // (9)
}

int data=1;
std::continuation f_ct= // (3)
    std::callcc(f,data); // (0)
data=f_ct.get_data<int>();
std::cout << "f: returned first time: " << data << std::endl;
f_ct = // (7)
```

*As stated in [Exceptions](#), if there is no matching `catch` clause in that context, `std::terminate()` is called.

```

    f_ct.resume(data+1); // (4)
data=f_ct.get_data<int>();
std::cout << "f: returned second time: " << data << std::endl;
f_ct = // (13)
    f_ct.resume_with(g,data+1); // (8)
data=f_ct.get_data<int>();
std::cout << "f: returned third time: " << data << std::endl;

```

output:

```

f: entered first time: 1
f: returned first time: 2
f: entered second time: 3
f: returned second time: 4
g: entered: 5
f: entered third time: -1

```

Control passes from (0) to (1) to (2), and so on.

The `f_ct.resume_with(g, data+1)` call at (8) passes control to `g()` on the context of `f()`.

The `return` statement at (9) causes the `resume()` call at (6) to return, executing the assignment at (10). The `int` returned by `g()` is accessed at (11).

Finally, `f()` returns its own `mc` variable, switching back to the main context.

There is one restriction on a function `fn()` passed to `resume_with()`: neither `fn()`, nor any function it calls, may perform a context switch back to the context that called `resume_with()` – whether directly, or indirectly via other contexts. `fn()` *must* return (or throw an exception) before `resume_with()` returns.

`std::callcc()` immediately enters new context `std::callcc()` creates a new context and immediately calls its passed *entry-function* on that new context.

This is intentional for consistency with the *call/cc* facility in other languages.^{7,8}

Moreover, this behavior prevents a problematic usage. Suppose we had a `callcc_deferred()` which would create a new context but immediately return to its caller. The newly-created context would first be entered by calling `resume()` on the returned `std::continuation` instance.

```

std::continuation newcontext = std::callcc_deferred(entry_function);
newcontext = newcontext.resume();

```

But now consider this scenario:

```

std::continuation newcontext = std::callcc_deferred(entry_function);
newcontext = newcontext.resume_with(injected_function);

```

What should happen here?

`resume_with()` is supposed to call `injected_function()` as if it had been directly called by `entry_function()` – rather, by the context-switch operation most recently executed by `entry_function()`. But since `entry_function()` has never yet been entered, it hasn't executed any context-switch operation. Indeed, it does not yet have a stack frame.

Should `injected_function()` become the *entry-function* for `newcontext`, displacing `entry_function()` entirely?

That would encounter signature problems. The *entry-function* invoked by `std::callcc()` *must* return a `std::continuation`. Yet the `fn` passed to `resume_with()` returns – not a `std::continuation` – but arbitrary data to be retrieved by the suspended context!

With the present API, to quickly resume the caller's context rather than prioritizing the new context, the *entry-function* passed to `std::callcc()` can immediately context-switch back to its caller by calling `resume()` on its passed-in `std::continuation`:

```

std::continuation entry_function(std::continuation&& caller) {
    caller = caller.resume();
}

```

```

    // ...
}

```

A more generic wrapper for that behavior could look something like this:

```

template <typename Fn>
std::continuation suspend_immediately(std::continuation&& caller) {
    Fn fn = caller.get_data<Fn>();
    return fn(caller.resume());
}

template <typename Fn>
std::continuation callcc_deferred(Fn&& fn) {
    return std::callcc(suspend_immediately<Fn>, std::forward<Fn>(fn));
}

```

Note that since `suspend_immediately()` *has* been entered, it is perfectly valid for the caller of `callcc_deferred()` to call `resume_with()` on the returned `std::continuation`.

Stack destruction On construction of a context with `std::callcc()` a stack is allocated. If the *entry-function* returns, the stack will be destroyed. If the function has not yet returned and the (destructor) of the `std::continuation` instance representing that context is called, the stack will be unwound and destroyed.

For this purpose member-function `resume_with()` is called with `std::unwind_context()` as argument. The execution context will be temporarily resumed and `std::unwind_context()` is invoked. Function `std::unwind_context()` throws exception `std::unwind_exception`.^{*} The exception is caught by the first frame on the stack: the one created by `std::callcc()`. Control is switched back to the context that called `~continuation()` and the stack gets deallocated.

The `StackAllocator`'s deallocate operation is called on the context that invoked `~continuation()`.

The stack on which `main()` is executed, as well as the stack implicitly created by `std::thread`'s constructor, is allocated by the operating system. Such stacks are recognized by `std::continuation`, and are not deallocated by its destructor.

Stack allocators are used to create stacks.

Stack allocators might implement arbitrary stack strategies. For instance, a stack allocator might append a guard page at the end of the stack, or cache stacks for reuse, or create stacks that grow on demand.

Because stack allocators are provided by the implementation, and are only used as parameters of `std::callcc()`, the `StackAllocator` concept is an implementation detail, used only by the internal mechanisms of the *call/cc* implementation. Different implementations might use different `StackAllocator` concepts.

However, when an implementation provides a stack allocator matching one of the descriptions below, it should use the specified name.

Possible types of stack allocators:

- `protected_fixedsize`: The constructor accepts a `size_t` parameter. This stack allocator constructs a contiguous stack of specified size, appending a guard page at the end to protect against overflow. If the guard page is accessed (read or write operation), a segmentation fault/access violation is generated by the operating system.
- `fixedsize`: The constructor accepts a `size_t` parameter. This stack allocator constructs a contiguous stack of specified size. In contrast to `protected_fixedsize`, it does not append a guard page. The memory is simply managed by `std::malloc()` and `std::free()`, avoiding kernel involvement.
- `segmented`: The constructor accepts a `size_t` parameter. This stack allocator creates a segmented stack⁹ with the specified initial size, which grows on demand.[†]

It is expected that the `StackAllocator`'s allocation operation will run in the context of the `std::callcc()` call (before control is passed to the new context), and that the `StackAllocator`'s deallocation operation will run in the context of the

^{*}`std::unwind_exception` binds an instance of `std::continuation` that represents the continuation that called `resume_with()`.

[†]An implementation of the segmented `StackAllocator` necessarily interacts with the C++ runtime. For instance, with gcc, the `Boost.Context`¹² library invokes the `__splitstack_makecontext()` and `__splitstack_releasecontext()` intrinsic functions.^{10,11}

`~continuation()` call (after control returns from the destroyed context). No special constraints need apply to either operation.

Performance of *call/cc*

On modern architectures suspending/resuming continuations takes very few CPU cycles. *

* `callcc()` from `boost.context` takes 18 CPU cycles on Intel E5 2620 v4, SYS V.

API

std::continuation declaration of class std::continuation

```
class continuation {
public:
    continuation() noexcept;
    ~continuation();
    continuation( continuation && other) noexcept;
    continuation & operator=( continuation && other) noexcept;
    continuation( continuation const& other) noexcept = delete;
    continuation & operator=( continuation const& other) noexcept = delete;

    template< typename ... Arg >
    continuation resume( Arg ... arg);
    template< typename Fn, typename ... Arg >
    continuation resume_with( Fn && fn, Arg ... arg);

    bool data_available() const noexcept;
    template< typename ... Arg >
    <unspecified> get_data();

    bool any_thread() const noexcept;

    explicit operator bool() const noexcept;
    bool operator!() const noexcept;
    bool operator==( continuation const& other) const noexcept;
    bool operator!=( continuation const& other) const noexcept;
    bool operator<( continuation const& other) const noexcept;
    bool operator>( continuation const& other) const noexcept;
    bool operator<=( continuation const& other) const noexcept;
    bool operator>=( continuation const& other) const noexcept;
    void swap( continuation & other) noexcept;
};

template< typename Fn, typename ...Arg >
continuation callcc( Fn &&, Arg ...);

template< typename StackAlloc, typename Fn, typename ...Arg >
continuation callcc( std::allocator_arg_t, StackAlloc, Fn &&, Arg ...);

void unwind_context( continuation && cont);

struct unwind_exception{};
```

member functions

(constructor) constructs new continuation

continuation() noexcept	(1)
continuation(continuation&& other)	(2)
continuation(const continuation& other)= delete	(3)

- 1) This constructor instantiates an invalid std::continuation. Its **operator bool()** returns **false**; its **operator!()** returns **true**.
- 2) moves underlying state to new std::continuation
- 3) copy constructor deleted

Notes

Every valid `std::continuation` instance is synthesized by the underlying facility – or move-constructed, or move-assigned, from another valid instance. There is no public `std::continuation` constructor that directly constructs a valid `std::continuation` instance.

The entry-function `fn` passed to `std::callcc()` is passed a synthesized `std::continuation` instance representing the suspended caller of `std::callcc()`.

The function `fn` passed to `resume_with()` is passed a synthesized `std::continuation` instance representing the suspended caller of `resume_with()`.

`std::callcc()` returns a synthesized `std::continuation` representing the previously-executing context, the context that suspended in order to resume the caller of `std::callcc()`. The returned `std::continuation` instance *might* represent the context created by `std::callcc()`, but need not: the context created by `std::callcc()` might have created (or resumed) yet another context, which might then have resumed the caller of `std::callcc()`.

Similarly, `resume()` returns a synthesized `std::continuation` instance representing the previously-executing context, the context that suspended in order to resume the caller of `resume()`.

Similarly, `resume_with()` returns a synthesized `std::continuation` instance representing the previously-executing context, the context that suspended in order to resume the caller of `resume_with()`.

(destructor) destroys a continuation

```
~continuation() (1)
```

- 1) destroys a `std::continuation` instance. If this instance represents a context of execution (`operator bool()` returns `true`), then the context of execution is destroyed too. Specifically, the stack is unwound by throwing `std::unwind_exception`.*

operator= moves the continuation object

```
continuation& operator=(continuation&& other) (1)
```

```
continuation& operator=(const continuation& other)=delete (2)
```

- 1) assigns the state of `other` to `*this` using move semantics
- 2) copy assignment operator deleted

Parameters

other another execution context to assign to this object

Return value

***this**

resume() resumes a continuation

```
template< typename ...Args >  
continuation resume( Args ... args) (1)
```

```
template< typename Fn, typename ...Args >  
continuation resume_with( Fn && fn, Args ... args) (2)
```

- 1) suspends the active context, resumes continuation `*this`
- 2) suspends the active context, resumes continuation `*this` but calls `fn()` in the resumed context (as if called by the suspended function)

Parameters

...args passed to the resumed continuation - see section [Passing data](#)

fn function invoked ontop of resumed continuation

* In a program in which exceptions are thrown, it is prudent to code a context's *entry-function* with a last-ditch `catch (...)` clause: in general, exceptions must *not* leak out of the *entry-function*. However, since stack unwinding is implemented by throwing an exception, a correct *entry-function* `try` statement must also `catch (std::unwind_exception const&)` and rethrow it.

Return value

continuation the returned instance represents the execution context (continuation) that has been suspended in order to resume the current context

Exceptions

- 1) `resume()` or `resume_with()` might throw `std::unwind_exception` if, while suspended, the calling context is destroyed
- 2) `resume()` or `resume_with()` might throw *any* exception if, while suspended:
 - some other context calls `resume_with()` to resume this suspended context
 - the function `fn` passed to `resume_with()` – or some function called by `fn` – throws an exception
- 3) Any exception thrown by the function `fn` passed to `resume_with()`, or any function called by `fn`, is thrown in the context referenced by `*this` rather than in the context of the caller of `resume_with()`.

Preconditions

- 1) `*this` represents a context of execution (`operator bool()` returns `true`)
- 2) the current `std::thread` is the same as the thread on which `*this` was originally launched

Postcondition

- 1) `*this` is invalidated (`operator bool()` returns `false`)

Notes

`resume()` preserves the execution context of the calling context as well as stack parts like *parameter list* and *return address*.^{*} Those data are restored if the calling context is resumed.

A suspended continuation can be destroyed. Its resources will be cleaned up at that time.

The returned continuation indicates whether the suspended context has terminated (returned from *entry-function*) via `operator bool()`. If the returned continuation has terminated, no data may be retrieved.

Because `resume()` invalidates the instance on which it is called, *no valid `std::continuation` instance ever represents the currently-running context*.

When calling `resume()`, it is conventional to replace the newly-invalidated instance – the instance on which `resume()` was called – with the new instance returned by that `resume()` call. This helps to avoid inadvertent calls to `resume()` on the old, invalidated instance.

An injected function `fn()` must accept `std::continuation&&`. However, instead of returning `std::continuation`, `fn()` must return a type corresponding to the args passed to the `resume_with()` call.

- If you call `ctx.resume_with(fn)` with no additional args, the return type of `fn()` is irrelevant: its return value is discarded.
- If you call `ctx.resume_with(fn, single_arg)`, then `fn()` must return `decltype(single_arg)`.
- If you call `ctx.resume_with(fn, multiple_args...)`, then `fn()` must return a `std::tuple` with corresponding types: specifically, `decltype(std::make_tuple(multiple_args...))`.

The value returned by an injected function `fn()` becomes available to the suspended function on the context represented by `*this`. It is up to `fn()` whether to return the same value(s) it retrieved from the `resume_with()` call.

- If you call `resume_with(fn)` with no additional args, then `data_available()` will return `false` even within the suspended function, which may not call `get_data()` at all.
- If you call `resume_with(fn, args...)` – with one or more args – then `data_available()` will return `true` within the suspended function. `get_data()` within the suspended function will retrieve the value returned by `fn()`.

Neither an injected function `fn()`, nor any function it calls, may perform a context switch back to the context that called `resume_with()` – whether directly, or indirectly via other contexts. `fn()` *must* return (or throw an exception) before `resume_with()` returns.

`data_available()` test if data are present

```
bool data_available() (1)
```

- 1) returns `true` if `std::callcc()` or `resume()` have been invoked with additional data as argument (`args`)

^{*}required only by some x86 ABIs

get_data() transfer of data

```
template< typename Arg >
Arg get_data() (1)
```

```
template< typename ...Args >
std::tuple< Args... > get_data() (2)
```

1) transfers single datum from continuation **this* into calling context

2) transfers multiple data from continuation **this* into calling context

Notes

The template argument(s) passed to `get_data()` must match in number and type the actual argument types passed to `std::callcc()`, `resume()` or `resume_with()`.

- If you call `std::callcc(fn), resume()` or `resume_with(fn)` with no additional arguments, then `data_available()` will return **false**: `fn()` may not call `get_data()` at all.
- If you call `std::callcc(fn, single_arg), resume(single_arg)` or `resume_with(fn, single_arg)`, then `data_available()` will return **true**, and if `fn()` calls `get_data()`, its template argument must match `get_data<decltype(single_arg)>()`.
- If you call `std::callcc(fn, multiple_args...), resume(multiple_args...)` or `resume_with(fn, multiple_args...)`, then `data_available()` will return **true**, and if `fn()` calls `get_data()`, its template arguments must match the types of `multiple_args`. Specifically, `get_data<Args...>()` returns `std::tuple<Args...>`; that `std::tuple` must be compatible with `std::make_tuple(multiple_args...)`.

operator bool test whether continuation is valid

```
explicit operator bool() const noexcept (1)
```

1) returns **true** if **this* represents a context of execution, **false** otherwise.

Notes

A `std::continuation` instance might not represent a context of execution for any of a number of reasons.

- It might have been default-constructed.
- It might have been assigned to another instance, or passed into a function.
`std::continuation` instances are move-only.
- It might already have been resumed – calling `resume()` invalidates the instance.
- The *entry-function* might have voluntarily terminated the context by returning.

The essential points:

- Regardless of the number of `std::continuation` declarations, exactly one `std::continuation` instance represents each suspended context.
- No `std::continuation` instance represents the currently-running context.

operator! test whether continuation is invalid

```
bool operator!() const noexcept (1)
```

1) returns **false** if **this* represents a context of execution, **true** otherwise.

Notes

See Notes for `operator bool()`.

(comparisons) establish an arbitrary total ordering for `std::continuation` instances

```
bool operator==(const continuation& other) const noexcept (1)
```

```
bool operator!=(const continuation& other) const noexcept (1)
```

```
bool operator<(const continuation& other) const noexcept (2)
```

```
bool operator>(const continuation& other) const noexcept (2)
```

```
bool operator<=(const continuation& other) const noexcept (2)
```

```
bool operator>=(const continuation& other) const noexcept (2)
```

- 1) Every invalid `std::continuation` instance compares equal to every other invalid instance. But because the running context is never represented by a valid `std::continuation` instance, and because every suspended context is represented by exactly one valid instance, *no valid instance can ever compare equal to any other valid instance*.
- 2) These comparisons establish an arbitrary total ordering of `std::continuation` instances, for example to store in ordered containers. (However, key lookup is meaningless, since you cannot construct a search key that would compare equal to any entry.) There is no significance to the relative order of two instances.

swap swaps two `std::continuation` instances

```
void swap(continuation& other) noexcept (1)
```

- 1) Exchanges the state of `*this` with `other`.

std::callcc() [LEWG: Some members of SG1 are not happy with the name `callcc`. While it does resemble facilities in other languages, they feel the proposed facility is not a close enough parallel and that the name might be misleading. Alternative names are invited.]

create and enter a new context, capturing the current execution context (the **current continuation**) in a `std::continuation` and passing it to the specified *entry-function*.

`std::callcc()` acts as a factory-function: it creates and starts a new execution context (stack etc.) and returns a continuation that represents the rest of the execution context's computation.

`std::callcc()` explicitly expresses the creation of a new execution context and the switch to the other execution path.

```
template< typename Fn, typename ...Args >
continuation callcc( Fn && fn, Args ...args) (1)
```

```
template< typename StackAlloc, typename Fn, typename ...Args >
continuation callcc( std::allocator_arg_t, StackAlloc salloc, Fn && fn, Args ...args) (2)
```

- 1) creates and immediately enters the new execution context (executing `fn`). The current execution context is suspended, wrapped in a continuation (`std::continuation`) and passed as argument to `fn`.
- 2) takes a callable as argument, requirements as for (1). The stack is constructed using `salloc` (see [Stack allocators](#)).

Parameters

fn callable (function, lambda, functor) executed in the new context; expected signature `continuation(continuation &&)`

...args data transferred to the new context - see section [Passing data](#)

Return value

continuation the returned instance represents the execution context (continuation) that was suspended in order to resume the current context

Preconditions

`std::callcc()` may only be called by code running on a `std::thread`, or on an execution agent created by a previous `std::callcc()` call.

Exceptions

- 1) calls `std::terminate` if an exception other than `std::unwind_exception` escapes *entry-function* `fn`
- 2) `std::callcc()` might throw `std::unwind_exception` if, while suspended, the calling context is destroyed
- 3) `std::callcc()` might throw *any* exception if, while suspended:
 - some other context calls `resume_with()` to resume this suspended context
 - the function `fn` passed to `resume_with()` – or some function called by `fn` – throws an exception
- 4) if *entry-function* `fn` contains a `catch(...)` clause, it should also catch and rethrow `std::unwind_exception`

Notes

`std::callcc()` preserves the execution context of the calling context as well as stack parts like *parameter list* and *return address*.^{*} Those data are restored if the calling context is resumed.

^{*}required only by some x86 ABIs

A suspended continuation can be destroyed. Its resources will be cleaned up at that time.

On return `fn` must specify a `std::continuation` to which execution control is transferred. Returning an invalid `std::continuation` instance (`operator bool()` returns `false`) invokes undefined behavior.

If an instance with valid state goes out of scope and its `fn` has not yet returned, the stack is unwound and deallocated.

There are a few different ways to terminate a given context without terminating the whole process, or engaging undefined behavior.

- Return a valid continuation from the *entry-function* `fn`.
- Call `std::unwind_context()` with a valid continuation. This throws a `std::unwind_exception` instance that binds that continuation.
- [LEWG: Should we publish the `std::unwind_exception` constructor that accepts `std::continuation`? Then another supported way would be to construct and throw `std::unwind_exception` “by hand,” which is what `std::unwind_context()` does internally.]
- Call `std::continuation::resume_with(std::unwind_context)`. This is what `~continuation()` does. Since `std::unwind_context()` accepts a `std::continuation`, and since `resume_with()` synthesizes a `std::continuation` and passes it to the subject function, this terminates the context referenced by the original `std::continuation` instance and switches back to the caller.
- Engage `~continuation()`: switch to some other context, which will receive a `std::continuation` instance representing the current context. Make that other context destroy the received `std::continuation` instance.

`std::unwind_context()` terminate the current running context, switching to the context represented by the passed `std::continuation`. This is like returning that `std::continuation` from the *entry-function*, but may be called from any function on that context.

```
void unwind_context( continuation && cont ) (1)
```

- 1) throws `std::unwind_exception`, binding the passed `std::continuation`. The running context’s first stack entry – the one created by `std::callcc()` – catches `std::unwind_exception`, extracts the bound `std::continuation` and terminates the current context by returning that `std::continuation`.

Parameters

`cont` the `std::continuation` to which to switch once the current context has terminated

Preconditions

- 1) `cont` must be valid (`operator bool()` returns `true`)

Return value

- 1) None: `std::unwind_context()` does not return

Exceptions

- 1) throws `std::unwind_exception`

`std::unwind_exception` is the exception used to unwind the stack referenced by a `std::continuation` being destroyed. It is thrown by `std::unwind_context()`. `std::unwind_exception` binds a `std::continuation` referencing the context to which control should be passed once the current context is unwound and destroyed.

Stack allocators are the means by which stacks with non-default properties may be requested by the caller of `std::callcc()`. The stack allocator concept is implementation-dependent; the means by which an implementation’s stack allocators communicate with `std::callcc()` is unspecified.

An implementation may provide zero or more stack allocators. However, a stack allocator with semantics matching any of the following must use the corresponding name.

`protected_fixedsize` The constructor accepts a `size_t` parameter. This stack allocator constructs a contiguous stack of specified size, appending a guard page at the end to protect against overflow. If the guard page is accessed (read or write operation), a segmentation fault/access violation is generated by the operating system.

`fixedsize` The constructor accepts a `size_t` parameter. This stack allocator constructs a contiguous stack of specified size. In contrast to `protected_fixedsize`, it does not append a guard page. The memory is simply managed by `std::malloc()` and `std::free()`, avoiding kernel involvement.

`segmented` The constructor accepts a `size_t` parameter. This stack allocator creates a segmented stack⁹ with the specified initial size, which grows on demand.

Additional notes

GPU *call/cc* as proposed in this paper is solely a CPU operation. It cannot be used to create a GPU execution agent, or to create or resume a CPU context from code running on a GPU.

SIMD does not interfere with *call/cc* and can be used as usual.

Of course, depending on the calling convention, some micro-processor registers dedicated to SIMD might be preserved and restored too ^{*}.

TLS *call/cc* is TLS-agnostic - best practice related to TLS applies to *call/cc* too.

call/cc only preserves and restores micro-processor registers at its invocation.

Migration between threads is forbidden. A `std::continuation` may only be resumed on the `std::thread` on which it was launched.

^{*}MS Windows x64 calling convention

References

- [1] [N3708: A proposal to add coroutines to the C++ standard library](#)
- [2] [N3985: A proposal to add coroutines to the C++ standard library](#)
- [3] [N4045: Library Foundations for Asynchronous Operations, Revision 2](#)
- [4] [N4649: Working Draft, Technical Specification on C++ Extensions for Coroutines](#)
- [5] [P0534R1: call/cc \(call-with-current-continuation\): A low-level API for stackful context switching](#)
- [6] [P0559R0: Operating principles for evolving C++](#)
- [7] [call/cc in Scheme](#)
- [8] [call/cc in Ruby](#)
- [9] [Split Stacks / GCC](#)
- [10] [Re: using split stacks](#)
- [11] [segmented_stack.hpp](#): [Boost.Context](#) implementation of [segmented_stack](#) [StackAllocator](#)
- [12] [Library *Boost.Context*](#)
- [13] [Library *Boost.Coroutine2*](#)
- [14] [Boost.Coroutine](#) example illustrating coroutine chaining
- [15] [Library *Boost.Fiber*](#)
- [16] [Boost.Fiber](#) performance using Skynet microbenchmark
- [17] [Library *Boost.Graph*](#)
- [18] [Boost.Graph](#) Visitor Concepts
- [19] [Library *Boost.Spirit*](#)
- [20] [Library *Boost.Spirit* Karma](#)
- [21] [C++Now 2014 talk: Coroutines, Fibers and Threads, Oh My](#)
- [22] [C++Now 2016 talk: Pulling Visitors](#)
- [23] [Boost developer mailing list: Lazy Spirit Generators?](#)